Assignment No 25

Name : Shivam Ramakant Deshmane

Domain : AI & ML

College : T.B Girwalkar Polytechnic Ambajogai

Q1.

```
Manual Hyperparameter Tuning Results:

n_neighbors = 3 --> Test Accuracy = 0.9800
n_neighbors = 5 --> Test Accuracy = 0.9800
n_neighbors = 7 --> Test Accuracy = 0.9800
n_neighbors = 11 --> Test Accuracy = 1.0000
n_neighbors = 13 --> Test Accuracy = 1.0000
n_neighbors = 15 --> Test Accuracy = 1.0000

--------------------------------
Best n_neighbors : 11
Highest Accuracy : 1.0
```

Q2.

```
Manual Hyperparameter Tuning Results:

C = 1, Kernel = linear --> Test Accuracy = 1.0000
C = 1, Kernel = rbf    --> Test Accuracy = 1.0000
C = 10, Kernel = linear --> Test Accuracy = 1.0000
C = 10, Kernel = rbf    --> Test Accuracy = 0.9800
C = 20, Kernel = linear --> Test Accuracy = 0.9800
C = 20, Kernel = rbf    --> Test Accuracy = 1.0000

--------------------------------
Best C        : 1
Best Kernel   : linear
Highest Accuracy : 1.0
```

Q3.

```
Grid Search Results:

   param_C param_kernel  mean_test_score
0        1       linear             0.95
1        1          rbf             0.93
2       10       linear             0.93
3       10          rbf             0.94
4       20       linear             0.93
5       20          rbf             0.95

--------------------------------
Best Parameters : {'C': 1, 'kernel': 'linear'}
Best CV Score   : 0.95
Test Accuracy   : 1.0
```

Q4.

```
Randomized Search Results:

   param_C param_kernel  mean_test_score
0        1       linear             0.95
1        1          rbf             0.93
2       20          rbf             0.95
3       10       linear             0.93
4       20       linear             0.93

--------------------------------
Best Parameters : {'kernel': 'linear', 'C': 1}
Best CV Score   : 0.95
Test Accuracy   : 1.0

Comparison with Grid Search (Q3)
--------------------------------
Grid Search Best Parameters : {'C': 1, 'kernel': 'linear'}
Grid Search Best Score      : 0.98 (approx.)
Random Search Best Parameters : {'kernel': 'linear', 'C': 1}
Random Search Best Score      : 0.95
```

Q5.

```python
# Step 1: Import Libraries
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Step 2: Load Iris Dataset
iris = load_iris()
X = iris.data
y = iris.target

# Step 3: Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    stratify=y,
    random_state=42
)

# Step 4: Create Random Forest Model
rf_model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

# Step 5: Train the Model
rf_model.fit(X_train, y_train)
```

```
    test_size=0.2,
    stratify=y,
    random_state=42
)

# Step 4: Create Random Forest Model
rf_model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

# Step 5: Train the Model
rf_model.fit(X_train, y_train)

# Step 6: Make Predictions
y_pred = rf_model.predict(X_test)

# Step 7: Calculate Accuracy
accuracy = accuracy_score(y_test, y_pred)

# Step 8: Print Accuracy
print("Random Forest Accuracy:", round(accuracy, 4))
```

```
Random Forest Accuracy: 0.9
```

Q6.

```
...  AdaBoost Accuracy          : 0.9333
     Gradient Boosting Accuracy : 0.9667
```

Q7.

```
# Step 1: Import Libraries
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from xgboost import XGBClassifier

# Step 2: Load Iris Dataset
iris = load_iris()
X = iris.data
y = iris.target

# Step 3: Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    stratify=y,
    random_state=42
)

# Step 4: Create XGBoost Model
xgb_model = XGBClassifier(
    n_estimators=100,
    learning_rate=0.1,
    max_depth=3,
    random_state=42,
```

```
    use_label_encoder=False,
    eval_metric='mlogloss'
)

# Step 5: Train the Model
xgb_model.fit(X_train, y_train)

# Step 6: Make Predictions
y_pred = xgb_model.predict(X_test)

# Step 7: Calculate Accuracy
accuracy = accuracy_score(y_test, y_pred)

# Step 8: Print Accuracy
print("XGBoost Accuracy:", round(accuracy, 4))
```

```
/usr/local/lib/python3.12/dist-packages/xgboost/training.py:200: UserWarning: [11:51:30] WARNING: /__w/xgboost
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)
XGBoost Accuracy: 0.9333
```

Q8.

```python
# Step 1: Import Libraries
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.ensemble import RandomForestClassifier

# Step 2: Load Iris Dataset
iris = load_iris()
X = iris.data
y = iris.target

# Step 3: Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    stratify=y,
    random_state=42
)

# Step 4: Create Random Forest Model
rf = RandomForestClassifier(random_state=42)

# Step 5: Define Parameter Grid
param_grid = {
    'n_estimators': [50, 100, 150],
    'max_depth': [3, 5, 7]
}
```

```python
# Step 6: Apply GridSearchCV
grid_search = GridSearchCV(
    estimator=rf,
    param_grid=param_grid,
    cv=5,
    scoring='accuracy'
)

# Step 7: Train the Model
grid_search.fit(X_train, y_train)

# Step 8: Print Best Parameters and Best Score
print("Best Parameters :", grid_search.best_params_)
print("Best CV Score   :", round(grid_search.best_score_, 4))

# Step 9: Test Accuracy (Optional)
test_accuracy = grid_search.score(X_test, y_test)
print("Test Accuracy   :", round(test_accuracy, 4))
```

```
Best Parameters : {'max_depth': 3, 'n_estimators': 50}
Best CV Score   : 0.9583
Test Accuracy   : 0.9667
```

Q9.

```
/usr/local/lib/python3.12/dist-packages/xgboost/training.py:200: UserWarning: [11:54:22] WARNING: /__w/xgboost
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)
Model Comparison:

              Model  Accuracy
0               SVM    1.0000
1     Random Forest    0.9000
2          AdaBoost    0.9333
3  Gradient Boosting    0.9667
4           XGBoost    0.9333

------------------------------
Best Model    : SVM
Best Accuracy : 1.0
```

Q10.

```
from sklearn.svm import SVC

for C in [1, 10, 20]:
    for kernel in ["linear", "rbf"]:
        model = SVC(C=C, kernel=kernel)
        model.fit(X_train, y_train)
        print(C, kernel, model.score(X_test, y_test))
```

```
1 linear 0.8532608695652174
1 rbf 0.6847826086956522
10 linear 0.8695652173913043
10 rbf 0.6902173913043478
20 linear 0.8695652173913043
20 rbf 0.7065217391304348
```

```python
from sklearn.model_selection import GridSearchCV
from sklearn.svm import SVC

param_grid={
    "C":[1,10,20],
    "kernel":["linear","rbf"]
}

grid=GridSearchCV(
    SVC(),
    param_grid,
    cv=5
)

grid.fit(X_train,y_train)

print(grid.best_params_)
print(grid.best_score_)
```

```
{'C': 10, 'kernel': 'linear'}
0.8719131488211722
```

```
from sklearn.model_selection import RandomizedSearchCV

random=RandomizedSearchCV(
    SVC(),
    param_grid,
    n_iter=5,
    cv=5,
    random_state=42
)

random.fit(X_train,y_train)

print(random.best_params_)
print(random.best_score_)
```

```
{'kernel': 'linear', 'C': 10}
0.8719131488211722
```

```python
from sklearn.ensemble import RandomForestClassifier

rf=RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

rf.fit(X_train,y_train)
```

```
RandomForestClassifier
RandomForestClassifier(random_state=42)
```

```python
from sklearn.ensemble import AdaBoostClassifier

ada=AdaBoostClassifier(
    n_estimators=100,
    random_state=42
)

ada.fit(X_train,y_train)
```

```
AdaBoostClassifier
AdaBoostClassifier(n_estimators=100, random_state=42)
```

```python
from sklearn.ensemble import GradientBoostingClassifier

gb=GradientBoostingClassifier(
    n_estimators=100,
    learning_rate=0.1,
    random_state=42
)

gb.fit(X_train,y_train)
```

```
GradientBoostingClassifier
GradientBoostingClassifier(random_state=42)
```

```python
from xgboost import XGBClassifier

xgb=XGBClassifier(
    n_estimators=100,
    learning_rate=0.1,
    max_depth=3,
    random_state=42,
    eval_metric="logloss"
)

xgb.fit(X_train,y_train)
```

```
XGBClassifier
XGBClassifier(base_score=None, booster=None, callbacks=None,
              colsample_bylevel=None, colsample_bynode=None,
              colsample_bytree=None, device=None, early_stopping_rounds=None,
              enable_categorical=True, eval_metric='logloss',
              feature_types=None, feature_weights=None, gamma=None,
              grow_policy=None, importance_type=None,
              interaction_constraints=None, learning_rate=0.1, max_bin=None,
              max_cat_threshold=None, max_cat_to_onehot=None,
              max_delta_step=None, max_depth=3, max_leaves=None,
              min_child_weight=None, missing=nan, monotone_constraints=None,
              multi_strategy=None, n_estimators=100, n_jobs=None,
```

Heart Disease Prediction

Age

45 - +

Cholesterol

250 - +

Resting BP

120 - +

Max Heart Rate

150 - +

Predict

Heart Disease Detected